

Introduction to Multimodal Foundation Models

Teaching machines to see and read in the same space

Teaching demonstration | 5th-semester B.Sc.

A simple-looking question



a photo of a dog

*"A golden retriever
playing in the park"*

How can a computer tell that a **picture** and a **sentence** are about *the same thing*?

A simple-looking question



a photo of a dog

*"A golden retriever
playing in the park"*

How can a computer tell that a **picture** and a **sentence** are about *the same thing*?

Classically, vision and language were **two separate worlds**.

A simple-looking question



a photo of a dog

*"A golden retriever
playing in the park"*

How can a computer tell that a **picture** and a **sentence** are about *the same thing*?

Classically, vision and language were **two separate worlds**.

Today: **GPT-4V**, **Gemini**, image search, robotics — systems that *see and read*.

A simple-looking question



a photo of a dog

*"A golden retriever
playing in the park"*

How can a computer tell that a **picture** and a **sentence** are about *the same thing*?

Classically, vision and language were **two separate worlds**.

Today: **GPT-4V**, **Gemini**, image search, robotics — systems that *see and read*.

Almost all of them are powered by **multimodal foundation models**. How?

Today's learning objectives

By the end of this lecture, you will be able to:

1. say what a **multimodal foundation model** is;
2. explain what an **embedding** is;
3. describe how **CLIP** learns a shared image–text space;
4. explain why this enables **zero-shot** recognition.

We will come back to this list at the very end.

First: what is a modality?

A **modality** is a distinct type of data — a channel through which the world is recorded: an **image** (pixels), **text** (symbols), **audio** (a waveform), and so on.

Multimodal = a system that works with **two or more** at once.



Image
pixel grid



Text
token sequence



Audio
waveform

First: what is a modality?

A **modality** is a distinct type of data — a channel through which the world is recorded: an **image** (pixels), **text** (symbols), **audio** (a waveform), and so on.

Multimodal = a system that works with **two or more** at once.



Image
pixel grid



Text
token sequence



Audio
waveform

To a computer these are **completely different numbers** — with different structure and statistics.

Not directly comparable.

Multimodal tasks are everywhere

Almost every task we care about **relates one modality to another**:

- **Captioning**: image $\rightarrow$ text
- **Visual QA**: image + question $\rightarrow$ answer
- **Text-to-image**: text $\rightarrow$ image
(DALL·E, Stable Diffusion)
- **Cross-modal search**: text $\leftrightarrow$ image
- **Video / audio**: frames + sound $\rightarrow$ labels
- **Robotics**: camera + sensors $\rightarrow$ action
- **Medical**: scan + clinical report
- **Assistants**: GPT-4V, Gemini

Multimodal tasks are everywhere

Almost every task we care about **relates one modality to another**:

- **Captioning**: image $\rightarrow$ text
- **Visual QA**: image + question $\rightarrow$ answer
- **Text-to-image**: text $\rightarrow$ image
(DALL·E, Stable Diffusion)
- **Cross-modal search**: text $\leftrightarrow$ image
- **Video / audio**: frames + sound $\rightarrow$ labels
- **Robotics**: camera + sensors $\rightarrow$ action
- **Medical**: scan + clinical report
- **Assistants**: GPT-4V, Gemini

All need one thing: a way to **compare and align** content *across* modalities. Let's zoom into two.



input: one image

Which caption matches the image?

- (a) *"a dog playing"* ✓
- (b) *"a cat"*
- (c) *"a red car"*

Captioning as a **multiple choice** — pick the caption that fits.

In focus 1: image captioning (image $\rightarrow$ text)



input: one image

Which caption matches the image?

- (a) *"a dog playing"* ✓
- (b) *"a cat"*
- (c) *"a red car"*

Captioning as a **multiple choice** — pick the caption that fits.

Easy for you — but a computer only sees **pixels** and **symbols**. To choose, it needs a **joint space**: a **common representation space for images and text**.



What position does the person in the pink shirt play?

- (a) striker
- (b) **goalie** ✓
- (c) defender

Two inputs — an **image** and a **text** question — read **together**.



What position does the person in the pink shirt play?

- (a) striker
- (b) **goalie** ✓
- (c) defender

Two inputs — an **image** and a **text** question — read **together**.

To match the question against the picture, both must live in one **joint space**: a **common representation space for images and text**.

What is a foundation model?

Foundation model

A single model **pretrained once** on **broad data at scale**, then **adapted to many downstream tasks** — not one bespoke model per task.

The old way

- hand-crafted features
(SIFT, bag-of-words)
- a separate pipeline *per task*
- **nothing transfers**

Now

- pretrain **one** model on the web
- reuse it across many tasks
- often **zero-shot** — no retraining

What is a foundation model?

Foundation model

A single model **pretrained once** on **broad data at scale**, then **adapted to many downstream tasks** — not one bespoke model per task.

The old way

- hand-crafted features
(SIFT, bag-of-words)
- a separate pipeline *per task*
- **nothing transfers**

Now

- pretrain **one** model on the web
- reuse it across many tasks
- often **zero-shot** — no retraining

You have met this idea for **text** already — a large language model is a **foundation model**.
Now we make it *multimodal*.

Now: multimodal foundation models

A **multimodal** foundation model brings **two or more modalities** into one pretrained model. They come in two families:

Representation — *align* modalities in one space:

CLIP (← our focus today)

Generative — *produce* one modality from another:

DALL·E, GPT-4V, Gemini

Now: multimodal foundation models

A **multimodal** foundation model brings **two or more modalities** into one pretrained model. They come in two families:

Representation — *align* modalities in one space:

CLIP (← our focus today)

Generative — *produce* one modality from another:

DALL·E, GPT-4V, Gemini

The main idea

Both families rest on a **shared representation** — one space where images and text become comparable. So that is what we build next.

Recap: embeddings (you have seen this for words)

An **embedding** turns an object into a vector. Geometry = meaning: similar things sit close together.

$$\text{cat} \rightarrow \begin{bmatrix} 0.31 \\ 1.18 \\ -0.70 \end{bmatrix} \quad \text{dog} \rightarrow \begin{bmatrix} 0.40 \\ 1.12 \\ -0.62 \end{bmatrix} \quad \text{car} \rightarrow \begin{bmatrix} -0.9 \\ 0.20 \\ 1.30 \end{bmatrix}$$

- cat and dog are **near**; car is far.
- Images can be embedded too: a CNN / ViT maps a picture to a vector.

Recap: embeddings (you have seen this for words)

An **embedding** turns an object into a vector. Geometry = meaning: **similar things sit close together**.

$$\text{cat} \rightarrow \begin{bmatrix} 0.31 \\ 1.18 \\ -0.70 \end{bmatrix} \quad \text{dog} \rightarrow \begin{bmatrix} 0.40 \\ 1.12 \\ -0.62 \end{bmatrix} \quad \text{car} \rightarrow \begin{bmatrix} -0.9 \\ 0.20 \\ 1.30 \end{bmatrix}$$

- cat and dog are **near**; car is far.
- Images can be embedded too: a CNN / ViT maps a picture to a vector.

*Open question: can pictures and words live in the **same** vector space?*

The goal: a joint multimodal space

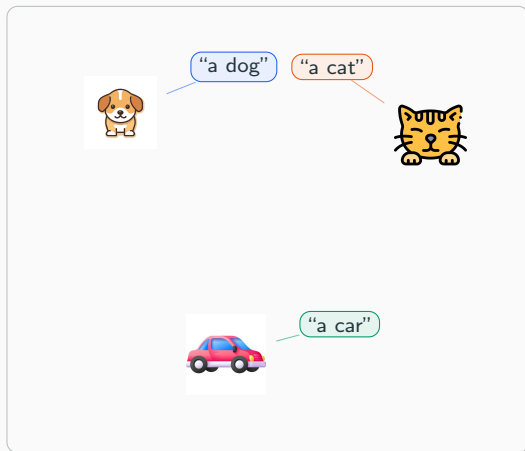
A **joint multimodal representation**

space: a single space where, for the **same concept**, the **image** and its **caption** land **close together** — and different concepts stay apart.

photo = image embedding

boxed text = caption embedding

a line links each **matching pair**



The goal: a joint multimodal space

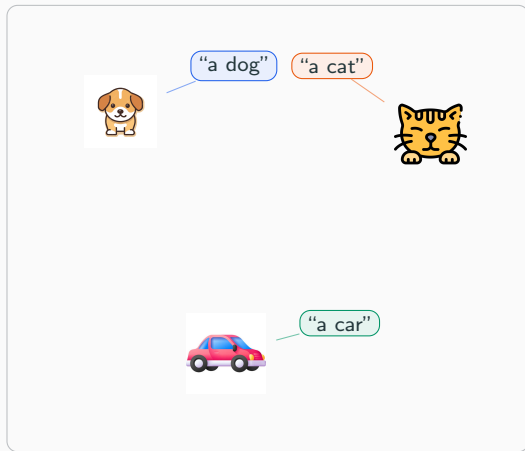
A **joint multimodal representation**

space: a single space where, for the **same concept**, the **image** and its **caption** land **close together** — and different concepts stay apart.

photo = image embedding

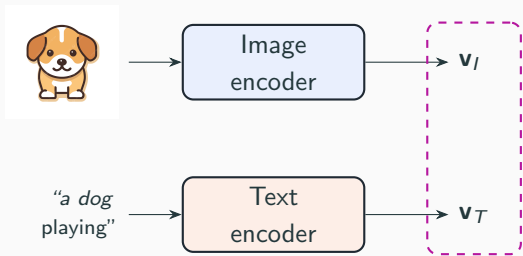
boxed text = caption embedding

a line links each **matching pair**



CLIP (2021) does exactly this, at scale — the first **multimodal foundation model** of its kind.

CLIP: two encoders, lots of (image, caption) pairs

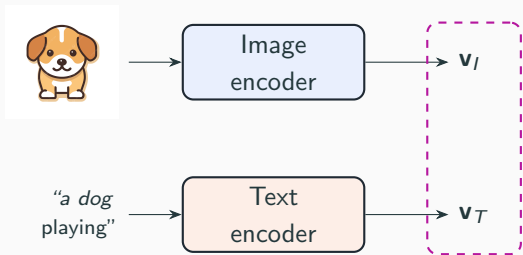


Training data: $\sim 400\text{M}$ (image, caption) pairs scraped from the web — no human labels.

Two networks, two vectors:

$$\mathbf{v}_I = f_{\text{img}}(\text{image}), \quad \mathbf{v}_T = f_{\text{txt}}(\text{text}).$$

CLIP: two encoders, lots of (image, caption) pairs



Training data: $\sim 400\text{M}$ (image, caption) pairs scraped from the web — no human labels.

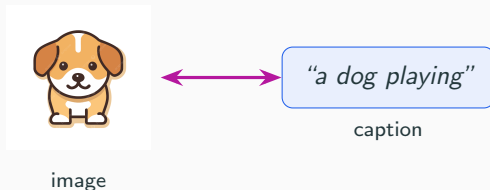
Two networks, two vectors:

$$\mathbf{v}_I = f_{\text{img}}(\text{image}), \quad \mathbf{v}_T = f_{\text{txt}}(\text{text}).$$

How do we **train** them to agree?

First: what is one training example?

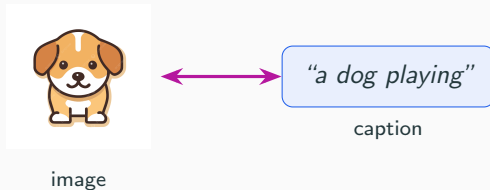
One training instance is simply an **(image, caption)** pair:



- The **caption itself** is the supervision — **not** a class label like “dog”.
- Nobody annotated it for us: it is the alt-text already sitting next to the image on the web.

First: what is one training example?

One training instance is simply an **(image, caption)** pair:

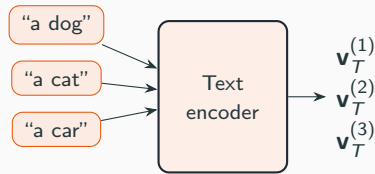
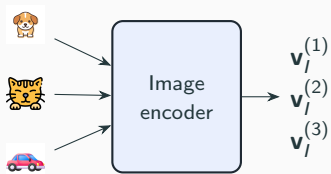


- The **caption itself** is the supervision — **not** a class label like “dog”.
- Nobody annotated it for us: it is the alt-text already sitting next to the image on the web.

That is exactly why CLIP could scrape $\sim 400\text{M}$ pairs — *supervision for free*.

Now a batch: a handful of pairs at once

Stack N such pairs (here $N=3$) and push them through the two encoders. Nothing exotic — exactly like ordinary mini-batch SGD.



Now a batch: a handful of pairs at once

Stack N such pairs (here $N=3$) and push them through the two encoders. Nothing exotic — exactly like ordinary mini-batch SGD.



No comparison has happened yet — we just hold six vectors: three image vectors and three caption vectors.

The trick: compare everything with everything

Compare **every image with every caption** — the cosine similarity of all $N \times N$ combinations:

$$\text{sim}(I, T) = \frac{\mathbf{v}_I^\top \mathbf{v}_T}{\|\mathbf{v}_I\| \|\mathbf{v}_T\|} \quad (\text{cosine: } -1 \dots 1)$$

A **similarity score** is just **how aligned two vectors are** in the shared space.

Every cell of the grid is one such score —
everything compared with everything.

	captions		
	dog	cat	car
images			

The trick: compare everything with everything

Compare **every image with every caption** — the cosine similarity of all $N \times N$ combinations:

$$\text{sim}(I, T) = \frac{\mathbf{v}_I^\top \mathbf{v}_T}{\|\mathbf{v}_I\| \|\mathbf{v}_T\|} \quad (\text{cosine: } -1 \dots 1)$$

A **similarity score** is just **how aligned two vectors are** in the shared space.

Every cell of the grid is one such score — **everything compared with everything**.

Take the **dog** image: we score it against every caption. Its true match is “a dog” — so we want *that* cell (**green**) to be the **largest**. Right now the scores are *nearly tied*.

		captions		
		dog	cat	car
images	dog	.31	.29	.28
	cat			
	car			

The trick: compare everything with everything

Compare **every image with every caption** — the cosine similarity of all $N \times N$ combinations:

$$\text{sim}(I, T) = \frac{\mathbf{v}_I^\top \mathbf{v}_T}{\|\mathbf{v}_I\| \|\mathbf{v}_T\|} \quad (\text{cosine: } -1 \dots 1)$$

A **similarity score** is just **how aligned two vectors are** in the shared space.

Every cell of the grid is one such score — **everything compared with everything**.

Take the **dog** image: we score it against every caption. Its true match is “a dog” — so we want *that* cell (**green**) to be the **largest**. Right now the scores are *nearly tied*.

		captions		
		dog	cat	car
images	dog	.31	.29	.28
	cat			
	car			

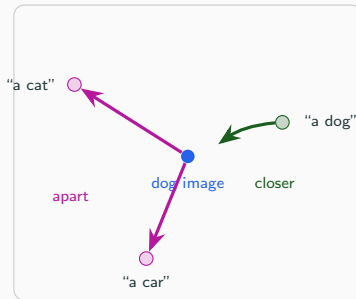
The question: how do you **train the parameters of the encoders** so the **correct** alignments (the true pairs) are **maximized**?

Training: the encoders

To maximise the correct alignments, we **update the parameters of the two encoders** (θ_I, θ_T) so the vectors move in the shared space:

- push **up** = update the encoders so the image and its **correct** caption move **closer**;
- push **down** = update them so the image and the **wrong** captions move **apart**.

Repeat over every batch — each update makes the correct cell score *higher* and the rest *lower*.



update θ : correct $\rightarrow$ closer, wrong $\rightarrow$ apart

Training: the encoders

To maximise the correct alignments, we **update the parameters of the two encoders** (θ_I, θ_T) so the vectors move in the shared space:

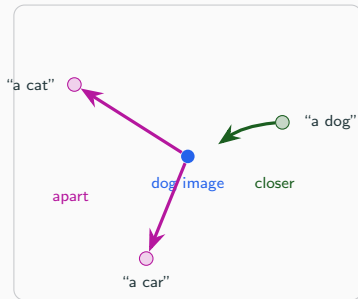
- push **up** = update the encoders so the image and its **correct** caption move **closer**;
- push **down** = update them so the image and the **wrong** captions move **apart**.

Repeat over every batch — each update makes the correct cell score *higher* and the rest *lower*.

dog row, now: .31 .29 .28 (nearly tied)

after training: **.92** .05 .03 (correct wins)

the exact update rule — gradient descent — comes *later in the course*.



update θ : correct $\rightarrow$ closer, wrong $\rightarrow$ apart

Do it for every row: contrastive learning

Now repeat that same contrast for **every row**
— and every column.

- every image pushes its **own** caption **up**
(the **green diagonal**);
- and pushes *all the others* **down**
(the **magenta** off-diagonal).

Correct *against* wrong, across the whole batch
— that is **contrastive learning**.

		captions		
		dog	cat	car
images	dog	+	-	-
	cat	-	+	-
	car	-	-	+

the whole **filled matrix**: a
green diagonal of true pairs

Do it for every row: contrastive learning

Now repeat that same contrast for **every row** — and every column.

- every image pushes its **own** caption **up** (the **green diagonal**);
- and pushes *all the others* **down** (the **magenta** off-diagonal).

Correct *against* wrong, across the whole batch — that is **contrastive learning**.

The negatives were already in the batch — **free**, no mining. With a single pair there would be *nothing to contrast against*: no batch, no signal.

		captions		
		dog	cat	car
images	dog	+	-	-
	cat	-	+	-
	car	-	-	+

the whole **filled matrix**: a green diagonal of true pairs

Which is the better representation space?

Same three pairs, two *different* trained models. Which similarity matrix is the better shared space — and why?

Space A

	dog	cat	car
images	.9	.1	.1
	.1	.9	.1
	.1	.1	.9

Space B

	dog	cat	car
images	.4	.4	.4
	.4	.4	.4
	.4	.4	.4

Which is the better representation space?

Same three pairs, two *different* trained models. Which similarity matrix is the better shared space — and why?

Space A

	dog	cat	car
images	.9	.1	.1
	.1	.9	.1
	.1	.1	.9

Space B

	dog	cat	car
images	.4	.4	.4
	.4	.4	.4
	.4	.4	.4

Space A — the diagonal must stand out

In A each image is **most similar to its own caption** and clearly *less* similar to the others: that **separation** is what lets us retrieve and classify. In B everything is equally similar — **no separation, no usable signal**, even though the cells look “warm”.

Training pushes the encoders toward A. *How* the parameters are actually learned — gradient descent — comes **later in the course**.

Reading the matrix: captioning & QA

Reading the matrix is simple — take **one image** (one row of the matrix), score it against a set of **text options**, and **pick the largest** cell.

Captioning (image → best caption)



"a dog playing"	0.29
"a cat"	0.11
"a red car"	0.08

⇒ caption: *"a dog playing"*

Reading the matrix: captioning & QA

Reading the matrix is simple — take **one image** (one row of the matrix), score it against a set of **text options**, and **pick the largest** cell.

Captioning (image → best caption)



"a dog playing"	0.29
-----------------	------

"a cat"	0.11
---------	------

"a red car"	0.08
-------------	------

⇒ caption: *"a dog playing"*

Visual QA (image + question → best answer)



"what position?"

goalie	0.31
--------	------

striker	0.12
---------	------

defender	0.09
----------	------

⇒ answer: **goalie**

Reading the matrix: captioning & QA

Reading the matrix is simple — take **one image** (one row of the matrix), score it against a set of **text options**, and **pick the largest** cell.

Captioning (image → best caption)



"a dog playing"	0.29
"a cat"	0.11
"a red car"	0.08

⇒ caption: *"a dog playing"*

Visual QA (image + question → best answer)



"what position?"

goalie	0.31
striker	0.12
defender	0.09

⇒ answer: **goalie**

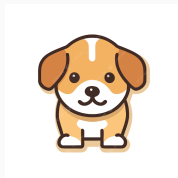
This is the *simplest*, retrieval-style version — choose from a list. Modern systems *generate* the words, but the scoring intuition is exactly this: **highest similarity wins**.

Zero-shot: new tasks it was never trained on

CLIP was trained *only* to match **(image, caption)** pairs — never to classify. Yet we can hand it a **brand-new set of labels at test time** and it just works — **no training examples, no fine-tuning**.

The recipe — turn any labels into text:

1. invent classes $\{y_1, \dots, y_K\}$ — even ones never seen in training;



a label set invented at test time:

“a golden retriever”

“a Siamese cat”

“a fire truck”

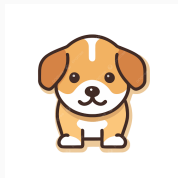
which label fits? — predict

Zero-shot: new tasks it was never trained on

CLIP was trained *only* to match (image, caption) pairs — never to classify. Yet we can hand it a **brand-new set of labels at test time** and it just works — **no training examples, no fine-tuning**.

The recipe — turn any labels into text:

1. invent classes $\{y_1, \dots, y_K\}$ — even ones never seen in training;
2. write each as a prompt: “a photo of a y_k ”;



a label set invented at test time:

“a golden retriever”

“a Siamese cat”

“a fire truck”

which label fits? — predict

Zero-shot: new tasks it was never trained on

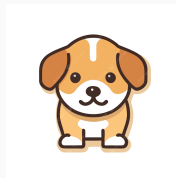
CLIP was trained *only* to match (image, caption) pairs — never to classify. Yet we can hand it a **brand-new set of labels at test time** and it just works — **no training examples, no fine-tuning**.

The recipe — turn any labels into text:

1. invent classes $\{y_1, \dots, y_K\}$ — even ones never seen in training;
2. write each as a prompt: “a photo of a y_k ”;
3. embed the image + prompts and **pick the most similar**:

$$\hat{y} = \arg \max_k \text{sim}(\mathbf{v}_I, \mathbf{v}_T^{(k)})$$

Same “read one row, take the max” as before — but the **classes are new**.



a label set invented at test time:

“a golden retriever”	.31
“a Siamese cat”	.10
“a fire truck”	.06

⇒ “a golden retriever” (highest similarity)

Zero-shot: new tasks it was never trained on

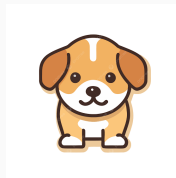
CLIP was trained *only* to match (image, caption) pairs — never to classify. Yet we can hand it a **brand-new set of labels at test time** and it just works — **no training examples, no fine-tuning**.

The recipe — turn any labels into text:

1. invent classes $\{y_1, \dots, y_K\}$ — even ones never seen in training;
2. write each as a prompt: “a photo of a y_k ”;
3. embed the image + prompts and **pick the most similar**:

$$\hat{y} = \arg \max_k \text{sim}(\mathbf{v}_I, \mathbf{v}_T^{(k)})$$

Same “read one row, take the max” as before — but the **classes are new**.



a label set invented at test time:

“a golden retriever”	.31
“a Siamese cat”	.10
“a fire truck”	.06

⇒ “a golden retriever” (highest similarity)

What you learned today

You can now explain

- what a **multimodal foundation model** is ✓
- what an **embedding** is — *geometry = meaning* ✓
- how **CLIP** learns a **shared space**:
compare every image with every caption,
then **update the encoders** so true pairs
align ✓
- why a **bright diagonal** lets us **match and retrieve** across images and text ✓

The one idea

A **shared representation space** where images and text become **comparable**.
Everything else — captioning, QA, retrieval — is just *reading the matrix*.

Next in the course: how the encoders themselves (CNN / ViT and Transformer) are built and trained.

Modern multimodal foundation models rest on one idea:
a shared representation where images and language
become comparable.

Thank you — questions?